

Ponerlo en el mapa: frontend, contenedores y despliegue moderno

Datos de la actividad

Número actividad	4
Docente	David Aldón Mínguez
Lengua de docencia	Castellano
Agrupación	Individual
Fecha de entrega	18-05-2026

Descripción de la actividad

La Actividad 4 introduce al estudiante en los fundamentos del desarrollo frontend y en la preparación de aplicaciones web para su ejecución y despliegue en entornos modernos. El objetivo es comprender cómo construir una interfaz web básica para la visualización de datos, cómo gestionar configuraciones mediante variables de entorno y cómo empaquetar una aplicación frontend dentro de un contenedor Docker.

A través de un caso práctico, el alumnado desarrolla una vista interactiva que permite explorar activos geolocalizados y peligros naturales mediante un mapa y filtros simples. Posteriormente, la aplicación se prepara para su ejecución en un contenedor y se documenta, de forma conceptual, el proceso de publicación de la imagen y su despliegue en un entorno cloud. La actividad puede completarse con implementaciones funcionales o mínimas, garantizando su independencia respecto a ejercicios anteriores.

Competencias

COMPETENCIAS (CE / CT)	RESULTADO DE APRENDIZAJE (RA)	INDICADORES DE EVALUACIÓN (IE)
CE8	RA5. Diseñar interfaces gráficas en un sitio web teniendo en cuenta las reglas de diseño y de la usabilidad.	IE5. El estudiante diseña una interfaz web funcional aplicando criterios básicos de diseño, usabilidad y presentación de la información.
CE2	RA6. Diseñar un sitio web que integre la interfaz de cliente, servidor y base de datos.	IE6. El estudiante integra una interfaz frontend con datos procedentes de una API o de una fuente de datos simulada.
CE3	RA7. Analizar y evaluar información desde una amplia gama de fuentes digitales utilizada para dar solución a un problema.	IE7. El estudiante valida el funcionamiento de la aplicación mediante pruebas básicas y verifica la correcta visualización de los datos.
CT4	RA8. Trabajar de forma responsable, ética y de acuerdo con el código deontológico de la profesión.	IE8. El estudiante aplica criterios de seguridad y responsabilidad en la configuración de la aplicación (variables de entorno y despliegue conceptual).

Contenido de la actividad

Contenidos	ODS
Desarrollo de interfaces web para la visualización de datos, incluyendo información geolocalizada en mapas.	ODS 13 – Acción por el clima La representación visual de datos territoriales permite analizar información por áreas y apoyar la toma de decisiones basada en datos.
Uso de tecnologías frontend modernas y principios básicos de diseño de interfaces.	ODS 4 – Educación de calidad El estudiante desarrolla competencias digitales avanzadas mediante la creación de interfaces web funcionales.
Preparación de la aplicación para su ejecución mediante contenedores y configuración por variables de entorno.	ODS 9 – Industria, innovación e infraestructura Se fomenta el uso de infraestructuras digitales modernas y portables.
Diseño conceptual del despliegue de aplicaciones web en entornos cloud.	ODS 8 – Trabajo decente y crecimiento económico La actividad aproxima al estudiante a prácticas profesionales habituales del sector tecnológico.

Descripción de las tareas

Frontend, Mapa de Riesgos y Contenerización

✉ Correo de Marta Ibarra — Frontend Lead (08:32 AM)

Asunto: *De los datos al mapa: vamos a pintar tus riesgos en pantalla*

Hola,

Hasta ahora has visto cómo procesamos y exponemos datos en DataSphere. Pero nuestros analistas no miran tablas: miran **pantallas** y, sobre todo, **mapas**.

En esta actividad te integrarás brevemente con el equipo de frontend para construir una **vista mínima de análisis de riesgos** y, después, prepararás esa aplicación para ejecutarse dentro de un **contenedor Docker** y documentarás cómo se podría desplegar en un entorno real (por ejemplo, publicando la imagen en un registro y desplegándola en la nube).

No necesitas haber completado las actividades anteriores:

- puedes trabajar con datos en memoria,
- puedes usar un mapa sencillo,
- y no es obligatorio tener un backend real funcionando.

Lo importante es que entiendas **cómo se construye una mínima interfaz, cómo se configura mediante variables de entorno y cómo se empaqueta para desplegarse**.

— Marta

Frontend Lead, DataSphere

🔧 Herramientas necesarias para completar la actividad

Para realizar correctamente la Actividad 4, el estudiante deberá utilizar un conjunto de herramientas habituales en el desarrollo de aplicaciones web modernas. Todas las herramientas indicadas son **gratuitas** y de uso ampliamente extendido en entornos profesionales.

1. Entorno de desarrollo

- **Visual Studio Code** (recomendado)
Editor de código ligero y multiplataforma, adecuado para el desarrollo frontend, la edición de ficheros de configuración y la creación de Dockerfiles.



Descarga: <https://code.visualstudio.com>

- **Node.js (LTS)**

Necesario para ejecutar y construir aplicaciones frontend modernas basadas en JavaScript y React.

Incluye npm para la gestión de dependencias.

Descarga: <https://nodejs.org>

2. Desarrollo frontend

- **React** (recomendado)

Biblioteca JavaScript para la construcción de interfaces de usuario basadas en componentes. Puede utilizarse mediante herramientas como Vite o Create React App.

- **Librería de mapas (una de las siguientes):**

- **Leaflet + OpenStreetMap** (recomendada)

Solución gratuita y sin necesidad de cuenta para mostrar mapas y marcadores geográficos.

<https://leafletjs.com>

- **Google Maps JavaScript API** (opcional)

Puede utilizarse si el estudiante ya dispone de una clave gratuita; no es obligatoria.

- **HTML + JavaScript**

Alternativa aceptada para la implementación mínima del frontend (con limitaciones en la evaluación).

3. Variables de entorno y configuración

- **Ficheros .env**

Utilizados para definir variables de entorno como URLs de servicios, nombre del entorno o configuraciones no sensibles.

El uso de variables de entorno permite separar configuración y código fuente.

- **Gestión de variables según entorno**

- VITE_ (Vite)

- REACT_APP_ (Create React App)

4. Contenerización

- **Docker Desktop**

Herramienta para crear, ejecutar y probar contenedores Docker de forma local.

Descarga: <https://www.docker.com/products/docker-desktop>

- **Dockerfile**

Archivo de texto que describe cómo construir una imagen de contenedor para el frontend.

Se utilizará tanto en versiones funcionales como en versiones mínimas (Hello World).

5. Diseño y diagramación

- **draw.io / diagrams.net**

Herramienta web gratuita para la creación de diagramas de arquitectura y flujos de despliegue. No requiere instalación.

Acceso: <https://app.diagrams.net>

6. Herramientas opcionales

- **Navegador web moderno** (Chrome, Firefox, Edge)

Para visualizar la aplicación frontend y comprobar el funcionamiento del mapa.

- **Docker Hub** (opcional)

Registro público de imágenes Docker utilizado únicamente a nivel conceptual en la Tarea 3. No es obligatorio crear una cuenta ni subir imágenes reales.



Objetivo

Construir una pequeña interfaz web que permita:

- visualizar activos en un listado,
- filtrar por tipo de peligro (*hazard*),
- mostrar sobre un mapa los activos que cumplen una condición (por ejemplo, terremoto con cierta severidad),
- y gestionar parte de la configuración mediante **variables de entorno** (por ejemplo, la URL de una API o el nombre del entorno).

Datos de ejemplo (array en memoria)

Puedes trabajar con un array de objetos como este (puedes ampliarlo si quieres):

```
[
  {
    "name": "Planta Norte 1",
    "latitude": 41.120,
    "longitude": 2.100,
    "regionCode": "ES-CT",
    "hazard": "Tornado",
    "severity": 3,
    "probabilityScore": 2,
    "assetValue": 1200000
  },
  {
    "name": "Estación Sur",
    "latitude": 40.250,
    "longitude": -3.700,
    "regionCode": "ES-MD",
    "hazard": "Earthquake",
    "severity": 4,
    "probabilityScore": 4,
    "assetValue": 800000
  },
  {
    "name": "Centro Logístico Este",
    "latitude": 39.470,
    "longitude": -0.380,
    "regionCode": "ES-VC",
    "hazard": "Rainstorm",
    "severity": 2,
    "probabilityScore": 3,
    "assetValue": 450000
  },
  {
    "name": "Planta Oeste",
    "latitude": 43.260,
    "longitude": -2.935,
    "regionCode": "ES-PV",
    "hazard": "Windstorm",
    "severity": 5,
    "probabilityScore": 4,
    "assetValue": 1500000
  }
]
```



Qué se espera de la UI (look & feel)

Tu interfaz debe tener, como mínimo:



1. **Un listado de activos**, mostrando:
 - o nombre,
 - o hazard,
 - o severity,
 - o regionCode,
 - o assetValue (formateado mínimamente).
2. **Un selector de hazard** (por ejemplo, un `<select>` o dropdown):
 - o al elegir un tipo de peligro (Tornado, Earthquake, etc.),
 - o el listado se filtra para mostrar solo los activos relacionados.
3. **Un mapa** donde se representen los activos filtrados:
 - o cada activo debe mostrarse como un punto o marcador en sus coordenadas (lat, long),
 - o al cambiar el hazard, deben actualizarse los puntos visibles.

Para el mapa puedes usar, por ejemplo:

- **React + Leaflet (OpenStreetMap)**
(recomendado: *no requiere cuenta de pago*)
- o Google Maps si el alumno ya tiene cuenta y key configurada, pero no es obligatorio.

👉 No es necesario implementar interacción avanzada: basta con que se visualicen puntos en el mapa según el filtro.

🔧 Tecnologías

- **Opción recomendada (100% de la nota de esta tarea):**
React (con Vite, Create React App, Next, etc.) + alguna librería de mapas (Leaflet, Google Maps, etc.).
- **Opción alternativa (máx. 70% de la nota de esta tarea):**
HTML + JavaScript (o jQuery) + librería de mapas.
(Se aceptará, pero se indicará al estudiante que la opción principal evaluada al 100% es React.)

🔑 Variables de entorno

Incluye al menos 1–2 variables de entorno, por ejemplo:

- VITE_API_URL o REACT_APP_API_URL con una URL dummy,
- VITE_ENVIRONMENT_NAME (ej. development, production-demo).

No es necesario que la API exista; basta con **leer y mostrar** el valor en la UI o en la consola (por ejemplo, en el pie de página mostrar “Entorno: development-demo”).

📁 Entregables Tarea 1

- Captura del **frontend funcionando**, donde se vea:
 - o listado de activos
 - o selector de hazard
 - o mapa con puntos
 - o partes relevantes del código implementado
 - o tu nombre visible en la UI (por ejemplo, en el header o footer)
- Captura del fichero `.env` (con valores dummy) o del lugar donde configuras las variables de entorno (sin credenciales reales).
- Explicación de cómo funciona el filtro y qué representa el mapa.

📝 Frase sugerida:

“Esta interfaz permite explorar visualmente activos y peligros naturales, filtrando por hazard y mostrando en el mapa los puntos de riesgo.”

■ TAREA 2 — Contenerización de la aplicación frontend

Aquí te incorporas un momento al equipo de “Platform & Runtime”.

🎯 Objetivo

Crear un **Dockerfile** que permita ejecutar tu frontend dentro de un contenedor.

Si no has podido completar la Tarea 1, podrás hacer una versión mínima con un HTML “Hello DataSphere”.



● Caso A — Has hecho la Tarea 1 (frontend de riesgos)

Crea un Dockerfile que construya y sirva tu aplicación frontend.

Ejemplo de flujo típico (no obligatorio, pero orientativo):

- FROM node:alpine
- copiar el código
- npm install
- npm run build
- usar nginx o un servidor estático para servir el build.

No se exige una configuración perfecta; lo importante es que se vea el intento de “build + serve” en el contenedor.

● Caso B — NO has hecho la Tarea 1

Crea un archivo index.html muy simple (“Hello DataSphere Frontend”) y un Dockerfile que sirva ese HTML desde un contenedor.

Ejemplo de idea:

- imagen base ligera (node, nginx o similar),
- copiar index.html,
- servirlo desde un servidor estático.

👉 En esta versión **no vale solo explicar**: tienen que crear el HTML y el Dockerfile real (aunque sea básico).

📁 Entregables Tarea 2

- Captura del Dockerfile.
- Captura de la ejecución (real o simulada) del contenedor:
 - puede ser una captura de la consola con docker build / docker run,
 - o una captura del navegador mostrando la app servida desde el contenedor.
- Tu nombre debe aparecer en alguna de las capturas (por ejemplo, en la propia página HTML o en el título de la ventana).

📄 Frase sugerida:

“Este contenedor empaqueta el frontend para que pueda ejecutarse de forma consistente en cualquier entorno.”

■ TAREA 3 — Diseño del despliegue conceptual (Docker Hub + Cloud)

Ahora entras en contacto con **Alex Ruiz**, un ingeniero de plataforma que te explica:

“Aunque no vayas a desplegar nada real, quiero ver si entiendes el viaje de tu imagen:

desarrollador → Dockerfile → imagen → registro → entorno de ejecución.”

🎯 Objetivo

Explicar, mediante **diagrama e instrucciones**, cómo subirías la imagen de tu frontend a un registro público (por ejemplo, Docker Hub) y cómo podrías desplegarla en una plataforma cloud (por ejemplo, AWS) de forma conceptual.

✓ Qué debes hacer

1. Diagrama (Draw.io / diagrams.net)

- Desarrollador / equipo frontend.
- Repositorio de código.
- docker build → creación de la imagen.
- Publicación en un registro (ej. Docker Hub).
- Entorno de ejecución en la nube (por ejemplo, “AWS ECS / EC2 / Elastic Beanstalk” o similar).
- Cliente final (navegador) accediendo al frontend.

2. Instrucciones escritas (lista de pasos)

- Pasos numerados explicando, en modo pseudo–procedimiento, algo como:
 1. Construir la imagen local.
 2. Loguearse en Docker Hub.



3. Hacer docker push a un repositorio público.
4. Crear un servicio en AWS (o similar) que use esa imagen.
5. Configurar variables de entorno en el servicio cloud.
6. Exponer un endpoint público para el frontend.

No se pide ejecutar nada real, solo mostrar que entiendes el flujo.

Entregables Tarea 3

- Diagrama del flujo de despliegue con tu nombre incluido.
- Lista numerada de pasos donde expliques cada uno de ellos

 Frase sugerida:

“Este flujo describe cómo se construiría, publicaría y desplegaría el frontend de DataSphere en un entorno cloud real utilizando imágenes Docker.”

ENTREGABLE FINAL (PDF)

El estudiante deberá entregar **un único documento en formato PDF** que incluya las siguientes evidencias:

1. Frontend

- Capturas de la interfaz funcionando:
 - listado de activos
 - selector de hazard
 - mapa con punto geolocalizados
 - código implementado
 - mapa con puntos geolocalizados.
- Evidencia del uso de variables de entorno (captura del .env o del código que las lee).
- Nombre del estudiante visible en la interfaz o en la captura.

2. Contenerización

- Captura del Dockerfile utilizado.
- Captura de la ejecución del contenedor o del frontend servido desde Docker (versión funcional o mínima).

3. Diseño del despliegue conceptual

- Diagrama del flujo de despliegue (build → imagen → registro → cloud(AWS como referencia, opcionalmente Azure)).
- Explicación escrita del proceso de publicación y despliegue (lista de pasos).

Evaluación

Criterio	Peso
Frontend (UI + mapa + variables de entorno)	50%
Contenerización del frontend (Docker)	25%
Diseño del despliegue conceptual	25%

Criterios generales de corrección

Se evaluará la **correcta aplicación de los conceptos, la claridad funcional y la coherencia con el**

caso planteado.

Las implementaciones en **React** permiten alcanzar la máxima puntuación del criterio de interfaz.
Las implementaciones alternativas (**HTML/JS**) podrán evaluarse si cumplen los requisitos funcionales.

1. Frontend e integración con datos (50%)

RA5. Diseñar interfaces gráficas teniendo en cuenta reglas de diseño y usabilidad.

IE5. El estudiante diseña una interfaz web funcional aplicando criterios básicos de diseño, usabilidad y presentación de la información.

RA6. Diseñar un sitio web que integre la interfaz de cliente, servidor y base de datos.

IE6. El estudiante integra una interfaz frontend con datos procedentes de una API o de una fuente de datos simulada.

Nivel	Qué debe verse
0–4.9	Interfaz poco funcional o difícil de usar y sin integración real con datos. La organización visual es confusa o la obtención de datos no funciona.
5–6.9	Interfaz básica que muestra información de forma limitada o con integración parcial de datos (por ejemplo datos simulados o visualización incompleta).
7–8.9	Interfaz clara y funcional que obtiene datos desde una API o fuente simulada y los muestra correctamente, aplicando criterios básicos de diseño y usabilidad.
9–10	Interfaz bien estructurada y fácil de usar que integra correctamente datos dinámicos desde la API, mostrando la información de forma clara y coherente con el funcionamiento del sistema..

2. Contenerización del frontend (Docker) (25%)

RA7. Analizar y evaluar información desde una amplia gama de fuentes digitales para resolver un problema.

IE7. El estudiante valida el funcionamiento de la aplicación mediante pruebas básicas y verifica la correcta visualización de los datos.

Nivel	Qué debe verse
0–4.9	No se presenta contenedor o la aplicación no funciona correctamente en el entorno propuesto.
5–6.9	Contenerización básica o parcial, con ejecución limitada o explicación conceptual del proceso.
7–8.9	Aplicación frontend ejecutándose correctamente en un contenedor Docker o entorno equivalente, con evidencias de funcionamiento.
9–10	Contenerización completa del frontend con evidencias claras de ejecución y verificación del correcto funcionamiento de la aplicación.

3. Diseño del despliegue conceptual y configuración segura (25%)

RA8. Trabajar de forma responsable, ética y de acuerdo con el código deontológico de la profesión.

IE8. El estudiante aplica criterios de seguridad y responsabilidad en la configuración de la aplicación

(variables de entorno y despliegue conceptual).

Nivel	Qué debe verse
0–4.9	No se consideran aspectos de configuración, seguridad o despliegue de la aplicación.
5–6.9	Descripción básica del despliegue o uso limitado de variables de entorno, con explicación superficial.
7–8.9	Propuesta clara de despliegue conceptual de la aplicación con uso adecuado de variables de entorno y buenas prácticas básicas.
9–10	Diseño completo del despliegue conceptual del sistema, aplicando buenas prácticas de configuración segura y gestión adecuada de variables de entorno.

Fuentes de información

Libros

- **Flanagan, D. (2020). *JavaScript: The Definitive Guide* (7th ed.). O'Reilly Media.**
Referencia fundamental para el desarrollo de aplicaciones frontend con JavaScript y la manipulación de datos en el navegador.
- **Banks, A., & Porcello, E. (2020). *Learning React* (2nd ed.). O'Reilly Media.**
Introducción práctica y clara al desarrollo de interfaces modernas basadas en React y componentes reutilizables.
- **Haverbeke, M. (2018). *Eloquent JavaScript* (3rd ed.). No Starch Press.**
Enfoque accesible para comprender la lógica y estructura de aplicaciones web frontend.
- **Merkel, D. (2014). *Docker: Lightweight Linux Containers for Consistent Development and Deployment*. Linux Journal.**
Texto de referencia para entender el uso de contenedores como mecanismo de empaquetado y portabilidad de aplicaciones.

Documentación técnica oficial

- **React Documentation**
<https://react.dev>
Documentación oficial para la construcción de interfaces de usuario basadas en componentes.
- **Leaflet Documentation**
<https://leafletjs.com>
Librería open source para la visualización de mapas interactivos basada en OpenStreetMap.
- **OpenStreetMap Project**
<https://www.openstreetmap.org>
Fuente de datos cartográficos abiertos utilizados en aplicaciones web geoespaciales.
- **Docker Documentation**
<https://docs.docker.com>
Documentación oficial sobre creación de imágenes, Dockerfile y ejecución de contenedores.

Recursos sobre despliegue y arquitectura

- **Richards, M., & Ford, N. (2020). *Fundamentals of Software Architecture*. O'Reilly Media.**
Referencia para comprender flujos de despliegue, arquitectura moderna y separación de responsabilidades.
- **AWS Architecture Overview**
<https://aws.amazon.com/architecture>
Visión general de arquitecturas cloud modernas utilizada a nivel conceptual en el diseño del despliegue.



Herramientas

- **Node.js** — <https://nodejs.org>
- **Visual Studio Code** — <https://code.visualstudio.com>
- **Docker Desktop** — <https://www.docker.com/products/docker-desktop>
- **draw.io (diagrams.net)** — <https://app.diagrams.net>

